TEAMAI

RAPPORT DU PROJET NLP 2

Détection et aide à l'interprétation d'anomalies dans les journaux HDFS

Master 2 Data & Intelligence Artificielle

Alain Evina · Bachar Ben Slama · Linda Boukhit · Nassim Mezzai

Version 0.1 : 16 octobre 2026

Document de travail : les résultats finaux et les validations collectives restent à compléter.

Sommaire prévisionnel

1. Introduction et cadrage du projet

2. Organisation et méthodologie

3. Présentation des solutions développées

4. Analyse technique comparative

5. Retour d’expérience

6. Étude de cinq outils novateurs

7. Conclusion et perspectives

Les chapitres 1 et 2 sont rédigés pour SCRUM-33. Les autres sont structurés et seront enrichis lors des prochains sprints.

1. Introduction et cadrage du projet

1.1 Contexte et problématique

Les systèmes informatiques distribués produisent de nombreux journaux d'événements (logs) qui retracent leur fonctionnement. Leur analyse permet de repérer des comportements inhabituels et d'aider à l'investigation d'incidents. Dans Hadoop Distributed File System (HDFS), les séquences d'événements associées aux blocs constituent une source d'information exploitable pour la détection d'anomalies.

Le projet TeamAI explore deux approches complémentaires : la classification automatique de traces HDFS à l'aide d'un modèle entraîné par l'équipe, puis l'aide à l'interprétation à partir d'un petit modèle de langage préentraîné, adapté et enrichi de documentation.

Problématique : comment détecter des anomalies dans les journaux HDFS et proposer une interprétation exploitable, tout en comparant la qualité, les ressources et le degré de maîtrise de deux approches d'intelligence artificielle ?

1.2 Objectifs

- Préparer un corpus HDFS reproductible et documenter ses partitions.
- Développer un classifieur normal/anomalie et le comparer à une baseline simple.
- Déployer une application web publique exécutant le modèle dans le navigateur, sans serveur d'inférence payant.
- Expérimenter un SLM local, son adaptation LoRA, un RAG documentaire et une organisation multi-agents.
- Comparer coût, latence, qualité, contrôle, explicabilité et dépendance fournisseur.

1.3 Périmètre

Volet A — application web	Volet B — application locale
Classification binaire de traces HDFS complètes. Modèle BiLSTM envisagé, export ONNX et inférence dans le navigateur.	Interprétation de traces à l'aide d'un SLM open-weights, d'une adaptation LoRA, de récupération documentaire (RAG) et d'agents spécialisés.
Entrée : logs HDFS au format documenté. Sortie : classe et résultat exportable.	Entrée : trace HDFS. Sortie : analyse distinguant faits, hypothèses et informations manquantes, avec sources.

Hors périmètre : supervision temps réel, collecte sur infrastructure de production, prise en charge de tous les formats de logs, correction automatique d'incidents et diagnostic causal garanti. Les deux applications restent utilisables indépendamment.

1.4 Hypothèses et contraintes

- Le corpus HDFS_v1 permet de construire des ensembles d'apprentissage, validation et test pertinents ; les déséquilibres et séquences répétées doivent être examinés.
- Le volet A doit respecter un budget d'hébergement nul et réaliser l'inférence côté client.

- Le volet B dépend du matériel disponible : la machine documentée pour les premiers essais possède un i5-1240P et 32 Go de RAM, sans GPU NVIDIA.
- Les évaluations doivent distinguer observations expérimentales, hypothèses et fonctionnalités encore prévues.

1.5 Risques et réponses prévues

Risque	Réponse prévue
Fuite de données ou performance surestimée	Séparer par bloc, contrôler les répétitions, réserver le test final et rapporter les résultats sur séquences inédites.
Incompatibilité ou lenteur du modèle web	Vérifier tôt la parité Python / ONNX Runtime Web, puis optimiser la taille et le chargement.
SLM trop lent ou trop gourmand	Mesurer CPU/RAM/latence ; tester la quantification et, si nécessaire, des ressources GPU pour l'adaptation.
Interprétations non fiables	Évaluer les faits et les sources ; intégrer RAG, contrôle et possibilité d'abstention.
Retards et dépendances entre tâches	Utiliser Jira, des responsables identifiés, des revues croisées et un journal des décisions.

1.6 Critères de réussite

- Application A publique, fonctionnelle, exécutant réellement un modèle dans le navigateur.
- Préparation des données reproductible et métriques finales sur un test réservé : précision, rappel, macro-F1 et matrice de confusion.
- Application B locale fournissant des interprétations sourcées et explicitant ses incertitudes.
- Adaptation effective du SLM, comparaison avant/après et mesure de l'apport du RAG et des agents.
- Scripts, configurations, décisions et limites suffisamment documentés pour reproduire les démonstrations.

Ces critères décrivent les objectifs du projet ; ils ne constituent pas des résultats déjà atteints.

2. Organisation et méthodologie

2.1 Équipe et responsabilités

Membre	Domaine principal	Contribution croisée
Nassim Mezzai	Audit HDFS, parsing, splits et évaluation	Relecture des données et du classifieur
Bachar Ben Slama	Classification, export ONNX, application web	Contrôle des données et tests navigateur

Alain Evina	SLM, adaptation LoRA, expérimentation et rapport	Relecture du RAG et des agents
Linda Boukhit	Corpus documentaire, RAG, orchestration et interface locale	Évaluation et coordination des interfaces

Cette répartition est issue du cadrage initial et de la feuille de route ; elle doit être ajustée aux contributions effectivement réalisées et validée par le groupe.

2.2 Planification et jalons

Jalon	Date	Attendu
J2	21/09/2026	Cadrage, étude des volets et organisation.
Mi-parcours	01/10/2026	Version minimale du volet A et première inférence locale.
J3	16/10/2026	Volet A fonctionnel, adaptation engagée, supports préparés, rapport commencé.
Rapport V1 (interne)	Fin novembre 2026	Consolidation du PDF et des expériences.
J4 — soutenance	07/12/2026	Présentation et démonstration des deux volets.

Le jalon de mi-parcours a été déclaré non tenu dans la feuille de route du 6 octobre. Un sprint de rattrapage a donc été planifié. Cette situation doit être décrite sans confondre les objectifs initiaux avec les livrables effectivement disponibles.

2.3 Pilotage, Git et traçabilité

L'équipe utilise Jira pour structurer les tâches en epics et sprints, avec responsable, relecteur et critère de fin. Le dépôt GitHub TeamAI conserve les scripts, la documentation et les preuves expérimentales. La branche main est destinée à la production ; dev sert à l'intégration ; les travaux sont développés sur des branches dédiées aux tickets et soumis par Pull Request.

Les décisions et les usages d'assistants IA sont consignés dans docs/projet/journal.md. La feuille de route rappelle que Jira constitue la source de vérité pour l'état des tâches. Le journal consulté ne contient pas encore de compte rendu de réunion attesté : les pratiques collectives devront donc être documentées avec précision plutôt que supposées.

2.4 Préparation du corpus HDFS

La préparation s'appuie sur HDFS_v1 (Loghub). Les lignes brutes sont associées à des identifiants de blocs (blk_) puis transformées en séquences d'événements, issues de 29 templates. L'audit, le parsing et la séparation apprentissage/validation/test ont fait l'objet de travaux spécifiques.

Le protocole retenu sur dev sépare les blocs de manière stratifiée ; il signale également les séquences inédites (unseen_seq) afin de compléter l'interprétation des métriques. Les exemples publics de l'application doivent provenir de la validation et non du test réservé. Le test final n'est pas utilisé pour régler les modèles.

2.5 Développement et évaluation du volet A

La démarche compare d'abord une baseline simple à un modèle BiLSTM, puis vérifie l'export ONNX et la cohérence de l'inférence entre Python et le navigateur. Les mesures pertinentes comprennent précision, rappel, macro-F1, faux positifs, faux négatifs, taille du modèle et latence côté client.

Une baseline par régression logistique est documentée dans SCRUM-24. Un essai de parité Python / navigateur a été mené sur un BiLSTM jouet à poids aléatoires : il vérifie la compatibilité technique de l'export, mais ne démontre pas les performances du modèle final entraîné.

2.6 Développement et évaluation du volet B

Le volet B commence par une inférence locale avec SmolLM2-1.7B-Instruct, puis examine la quantification, l'adaptation LoRA, l'intégration du RAG et l'intérêt d'agents de recherche, rédaction et contrôle. Les comparaisons prévues doivent utiliser des jeux réservés et un protocole comparable, en mesurant notamment la fidélité aux faits, la pertinence des sources, l'abstention, la latence et la mémoire.

Les premières expériences montrent des temps de génération élevés sur CPU et des interprétations incomplètes ou inventées. Un essai GGUF 4 bits a réduit la mémoire observée et amélioré le débit dans une configuration donnée, mais le journal précise que ces mesures ne constituent pas une comparaison contrôlée. Les conclusions devront être confirmées sur plusieurs traces.

2.7 Reproductibilité et limites

Chaque expérience doit documenter la date, le responsable, le code, les données, les paramètres, la machine, les résultats et les erreurs. Les métriques finales, la validation collective des choix et les retours d'expérience individuels seront ajoutés après vérification. Aucune conclusion générale ne doit être tirée d'une seule trace ou d'un essai de faisabilité.

3. Présentation des solutions développées

À compléter

4. Analyse technique comparative

À compléter

5. Retour d’expérience

À compléter

6. Cinq outils novateurs

À compléter

7. Conclusion et perspectives

À rédiger après consolidation des évaluations finales et de la comparaison des deux volets.